

BUILD SPEC · ONE SCREEN, ONE BEHAVIOUR

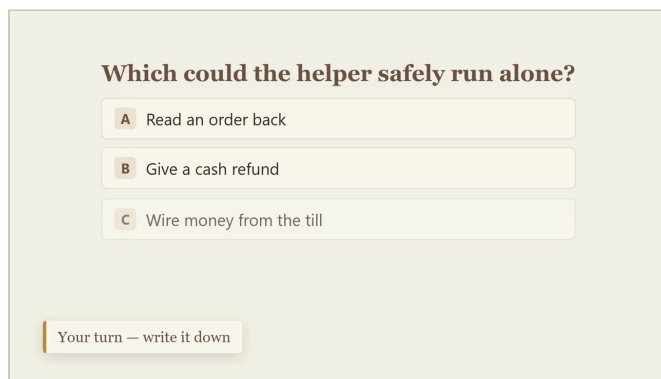
In-Video Question Checkpoint

Every Drawing Room lesson video contains a point where the narrator stops and asks the learner a question. Today that question is *painted into the video*, so nobody can answer it. This document specifies the replacement: **a popup the LMS draws over the player at an exact second**, which the learner must answer before the video continues.

LIVE DEMO	content-queen-production.up.railway.app/demo/quiz Press play, let it reach the gold marker, and answer. Every screenshot in this document was captured from that page.
LIVE API	content-queen-production.up.railway.app/api/v1 Deployed and serving. The index is public; the data routes need the bearer token on page 4. Because that token is in this document, do not post this file publicly .
TEST VIDEO	YouTube ID NkWLML1Uy7U · youtu.be/NkWLML1Uy7U Autonomy 01 — The Autonomy Spectrum · 2:11 · unlisted · one checkpoint, firing at 41.671s . Its API <code>videoId</code> is <code>autonomy-01-spectrum</code> .
YOUR SCOPE	One overlay component in the player, and one call to fetch its data. No change to how videos are stored, streamed or listed.

What this replaces

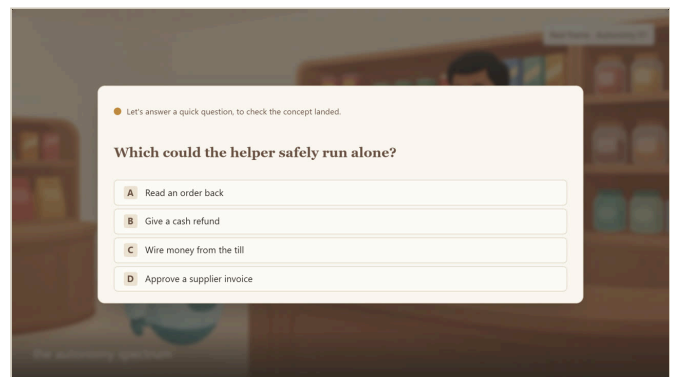
Both frames are the same video at **0:39**. Left is what learners get today.



TODAY

Painted into the video

The question is part of the picture, so there is nothing to click. Note the caption we were forced to write: **"Your turn — write it down."** The learner answers on paper, or more often not at all, and the video never knows either way.



PROPOSED

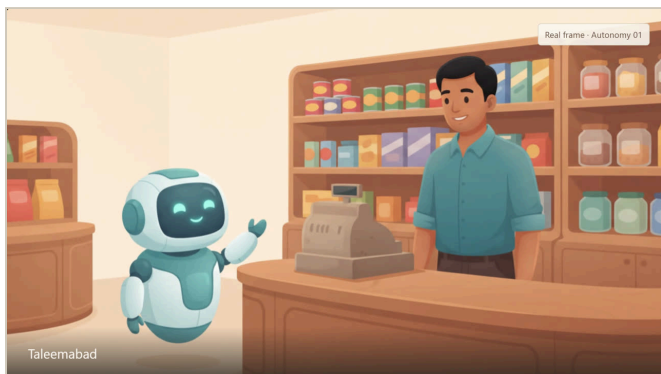
Drawn over the paused video

Same question, same wording, same house styling — but the LMS draws it, so the four options are real buttons. The learner answers in the player and is told immediately whether they were right.

The four states

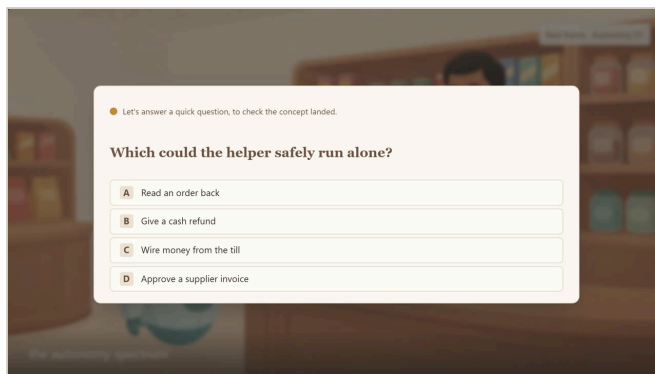
Everything the component has to render. All four captured from the working demo.

1 • PLAYING



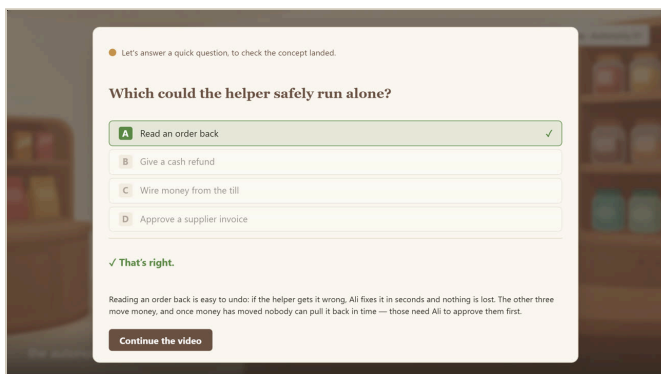
Before the checkpoint. Ordinary playback. The only hint is the marker on the timeline, which is optional — the learner does not need to know it is coming.

2 • QUESTION



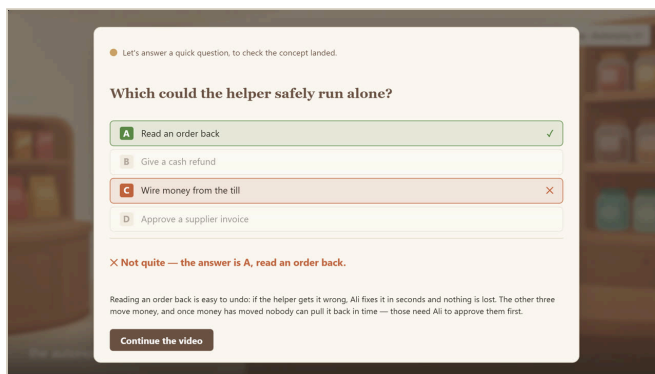
At 0:39 the video pauses itself and the popup opens over the dimmed frame. Four options, no submit button, no close button.

3 • CORRECT



Right answer. The chosen option turns green with a tick. The explanation still shows — a learner who guessed deserves the reasoning too.

4 • WRONG



Wrong answer. Their choice turns terracotta with a cross and the correct option turns green, both visible at once, so they can see exactly what they mixed up.

What happens, in order

- 01 **Playback reaches the checkpoint.** The player watches `currentTime` and fires at the second we supply. The video pauses itself — the learner should not have to pause it.
- 02 **The popup opens over the paused frame.** The frame stays visible and dimmed behind it, so the learner keeps the context they were just taught.
- 03 **The learner picks one of four options.** One click. No submit step, no confirmation.
- 04 **Right or wrong, immediately.** Correct: green tick. Wrong: their choice in terracotta, the correct answer revealed in green, then the explanation.
- 05 **The learner continues.** One button closes the popup and resumes from the same second. A wrong answer does not block them and is not scored — this is a comprehension check, not a test.

The endpoints

Deployed and serving. Base URL `https://content-queen-production.up.railway.app`.

ENDPOINT	AUTH	RETURNS
GET <code>/api/v1</code>	public	The surface, machine-readable: every route, an example URL for each, and whether the server is configured.
GET <code>/api/v1/videos</code>	bearer	The catalogue — every video that has at least one checkpoint. 17 today.
GET <code>/api/v1/videos/{videoId}/checkpoints</code>	bearer	The payload for one video. This is the only call you need to build the feature.
POST <code>/api/v1/videos/{videoId}/checkpoints/{id}/attempts</code>	bearer	Returns 501 — not implemented, deliberately. Record answers on your side; see page 7. It answers 501 rather than 404 so a caller gets a clear failure instead of silently losing data.
GET <code>/demo/quiz</code>	public	The interactive demo this document describes.

Authentication

The payload contains `correctIndex`, so it is not public — serving it openly would hand every learner the answer key. Send this token on every request to a `/api/v1` route except the index.

YOUR CREDENTIAL · TREAT AS A PASSWORD

HEADER **Authorization: Bearer <token>**

TOKEN **cq_o9UjsZ6_pknF10IUZwuZHM7H16MpJtDF**

This token is **read-only over lesson content** — it can fetch checkpoints and post an attempt. It cannot start work, change a video, or spend anything. **Do not put it in browser-side code** or commit it to a repository: anyone holding it can read the answer key. If it leaks, tell Aroma and it is replaced in one command, with no change on your side beyond the new value.

REQUEST · CURL

```
# paste both lines; the first is the token above
export CONTENT_API_TOKEN=cq_o9UjsZ6_pknF10IUZwuZHM7H16MpJtDF

curl -H "Authorization: Bearer $CONTENT_API_TOKEN" \
  https://content-queen-production.up.railway.app/api/v1/videos/autonomy-01-spectrum/checkpoints
```

Call this **from your server, not from the browser** — a page fetching it directly would ship the token to every learner. Fetch it server-side and hand your player only what it needs. Responses carry an **ETag**, so you can revalidate cheaply and get a **304** when nothing has changed.

One video's checkpoints

The real response for `autonomy-01-spectrum`. Nothing here is hand-maintained — it is read out of the video's own script and timed from its measured beat lengths, so a re-cut video cannot silently keep a stale question time.

GET /API/V1/VIDEOS/AUTONOMY-01-SPECTRUM/CHECKPOINTS · 200

```
{
  "videoId": "autonomy-01-spectrum",
  "series": "autonomy",
  "deliverable": "autonomy-01-spectrum_final.mp4",
  "timing": {
    "trusted": true,          // never fire a checkpoint when this is false
    "relativeTo": "autonomy-01-spectrum_final.mp4",
    "introOffsetSeconds": 2.6,
    "introOffsetSource": "brand-constant",
    "lessonSeconds": 126.26
  },
  "checkpoints": [
    {
      "id": "q1",
      "atSeconds": 41.671,    // fire here, in the delivered file
      "lessonAtSeconds": 39.071, // same moment, excluding the intro
      "preamble": "Let's answer a quick question, to check the concept landed.",
      "stem": "Which could the helper safely run alone?",
      "options": [
        "Read an order back",
        "Give a cash refund",
        "Wire money from the till",
        "Approve a supplier invoice"
      ],
      "correctIndex": 0,
      "explanation": "Easy to undo, so it runs free.",
      "beatId": "09",        // traceable back to the script
      "revealBeatId": "22"
    }
  ]
}
```

The timing rule that will bite you

Two numbers describe the same moment, and picking the wrong one makes every question fire early.

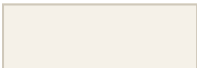
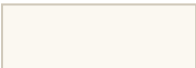
FIELD	MEANING
atSeconds	Use this one. Measured from the first frame of the delivered <code>*_final.mp4</code> , which opens with a 2.6 second brand intro.
lessonAtSeconds	The same moment measured from the lesson body, excluding that intro. Published for traceability only. Firing on it puts every question 2.6 seconds early.
timing.trusted	False means we could not establish the timing for that video. Do not fire its checkpoints — show the video without them rather than interrupting at the wrong moment.

Behaviour rules

SITUATION	REQUIRED BEHAVIOUR
Reaching atSeconds	Pause playback and open the popup automatically. Never wait for the learner to notice a button.
Answering twice	Not possible. All four options are disabled the moment one is chosen.
Trying to skip	No close button, no click-outside-to-dismiss, no Escape. The only exit is Continue the video , which appears only after answering.
Seeking past it	If a learner drags the timeline past an unanswered checkpoint, it should still fire — otherwise the check is trivially skippable.
Rewatching	Fire again. A rewatch is usually someone who wants another go at understanding it.
Keyboard & screen readers	Options reachable by Tab and answerable with keys 1–4; focus moves into the popup on open and to Continue after answering; the verdict is announced via aria-live .
Narrow player / mobile	The popup scales with the player and never overflows it. Sizes in the demo are set in container units for exactly this reason.
Where the checkpoint sits	Place it on a scene beat, not on one of our full-screen text cards, so the popup has a clean ground behind it.

Palette

Taken from the renderer's own stylesheet, so the popup matches the video rather than approximating it.

					
#F5F1E8 cream	#FBF8F1 card	#6B5344 heading	#C08A3E gold	#5C8A46 correct	#C0653E wrong

Open decisions

Four things still to settle. The first two are ours to fix; the last two are yours to tell us.

The explanation is currently one short line.

`explanation` comes from the video's REVEAL card, which was written to be read aloud in six words — for Autonomy 01 it is **“Easy to undo, so it runs free.”** That is thin for a learner who just got it wrong. We can author a longer popup-specific explanation per question; it does not change your code. `ours` · `say if you want it`

One explanation, or one per wrong option?

Today any wrong answer gets the same explanation. Per-option feedback teaches better and would add an `optionFeedback` array to the payload — a small change for you, three times the writing for us. `assumed: one` `shared`

Where do right and wrong answers get recorded?

On your side. We store nothing. This service holds no learner identity and has no database — an earlier draft of the `attempts` endpoint wrote to a file the host wipes on every deploy and answered “recorded”, which was untrue, so it now returns **501**. Your platform already owns the learner, the enrolment and the gradebook, so it is the system of record. If you later want us to receive attempts for content-quality analysis, say so and we will build a durable store rather than an endpoint that drops what it is sent. `settled 8 Sep` · `LMS is the record`

One question per video, or several?

`checkpoints` is an array, so several already work. Every video today has exactly one, about two-thirds through. Build the loop, not the single case. `assumed: one`, `extensible`